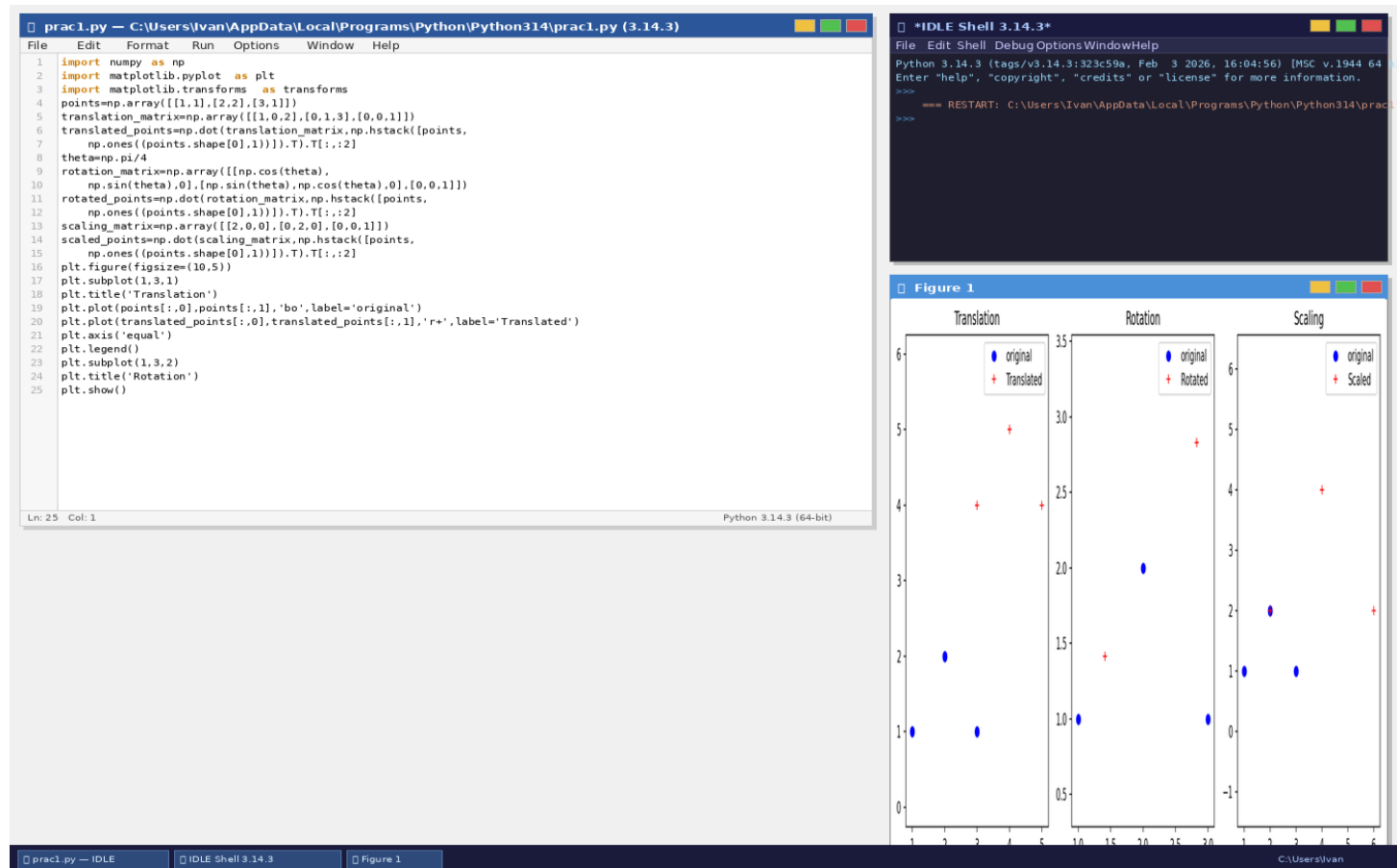


Practical No.	Name of the Practical	Date	Page No.	Signature
1.	Perform Geometric transformation	07/03/2026	1	
2.	Perform Image Stitching	07/03/2026	2	
3.	Perform Camera Calibration	07/03/2026	3	
4.	Face detection	14/03/2026	4	
5.	Object detection	14/03/2026	5	
6.	Pedestrian detection	20/03/2026	6	
7.	Perform Feature extraction using RANSAC	20/03/2026	7	
8.	Perform Colorization	20/03/2026	8	
9.	Perform Image matting and Composting	20/03/2026	9	

Practical 1 — Geometric Transformations

Practical 2 — Image Stitching

The screenshot displays a Python IDE with three windows: a script editor, a shell, and a window showing the stitched image.

Script Editor (prac2.py):

```
1 import cv2
2 import numpy as np
3
4 image1 = cv2.imread(
5     "C:/Users/Ivan/Desktop/msc 2025-26/stitching.jpg")
6 image2 = cv2.imread(
7     "C:/Users/Ivan/Desktop/msc 2025-26/stitching.jpg")
8 print("Image 1 shape:", image1.shape)
9 print("Image 2 shape:", image2.shape)
10
11 gray1 = cv2.cvtColor(image1, cv2.COLOR_BGR2GRAY)
12 gray2 = cv2.cvtColor(image2, cv2.COLOR_BGR2GRAY)
13
14 sift = cv2.SIFT_create()
15 keypoints1, descriptors1 = sift.detectAndCompute(gray1, None)
16 keypoints2, descriptors2 = sift.detectAndCompute(gray2, None)
17
18 matcher = cv2.BFMatcher()
19 matches = matcher.match(descriptors1, descriptors2)
20 matches = sorted(matches, key=lambda x: x.distance)
21
22 points1 = np.float32([keypoints1[m.queryIdx].pt
23     for m in matches]).reshape(-1, 1, 2)
24 print("Number of points in points1:", len(points1))
25
26 points2 = np.float32([keypoints2[m.trainIdx].pt
27     for m in matches]).reshape(-1, 1, 2)
28 print("Number of points in points2:", len(points2))
29
30 homography, _ = cv2.findHomography(points1, points2, cv2.RANSAC)
31 height, width = gray2.shape
32 stitched_image = cv2.warpPerspective(image1, homography, (width, height))
33 stitched_image[0:image2.shape[0], 0:image2.shape[1]] = image2
34
35 cv2.imshow('Stitched Image', stitched_image)
36 cv2.waitKey(0)
37 cv2.destroyAllWindows()
```

Shell (*IDLE Shell 3.14.3*):

```
Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:56) [MSC v.1944 64 bit (AMD64)]
Enter "help", "copyright", "credits" or "license()" for more information.
>>>
=== RESTART: C:\Users\Ivan\AppData\Local\Programs\Python\Python314\prac2.py ===
>>>
Image 1 shape: (300, 300, 3)
Image 2 shape: (300, 300, 3)
Number of points in points1: 49
Number of points in points2: 49
>>>
```

Stitched Image:

The window displays a 300x300 pixel image of a yellow smiley face on a black background, overlaid on a white grid. The smiley face is composed of two black dots for eyes and a curved line for a mouth.

Practical 3 — Camera Calibration

The image displays a Python script for camera calibration using OpenCV, running in an IDE. The script processes a chessboard image to find corners and calculate the camera matrix and distortion coefficients.

Python Script (prac3.py):

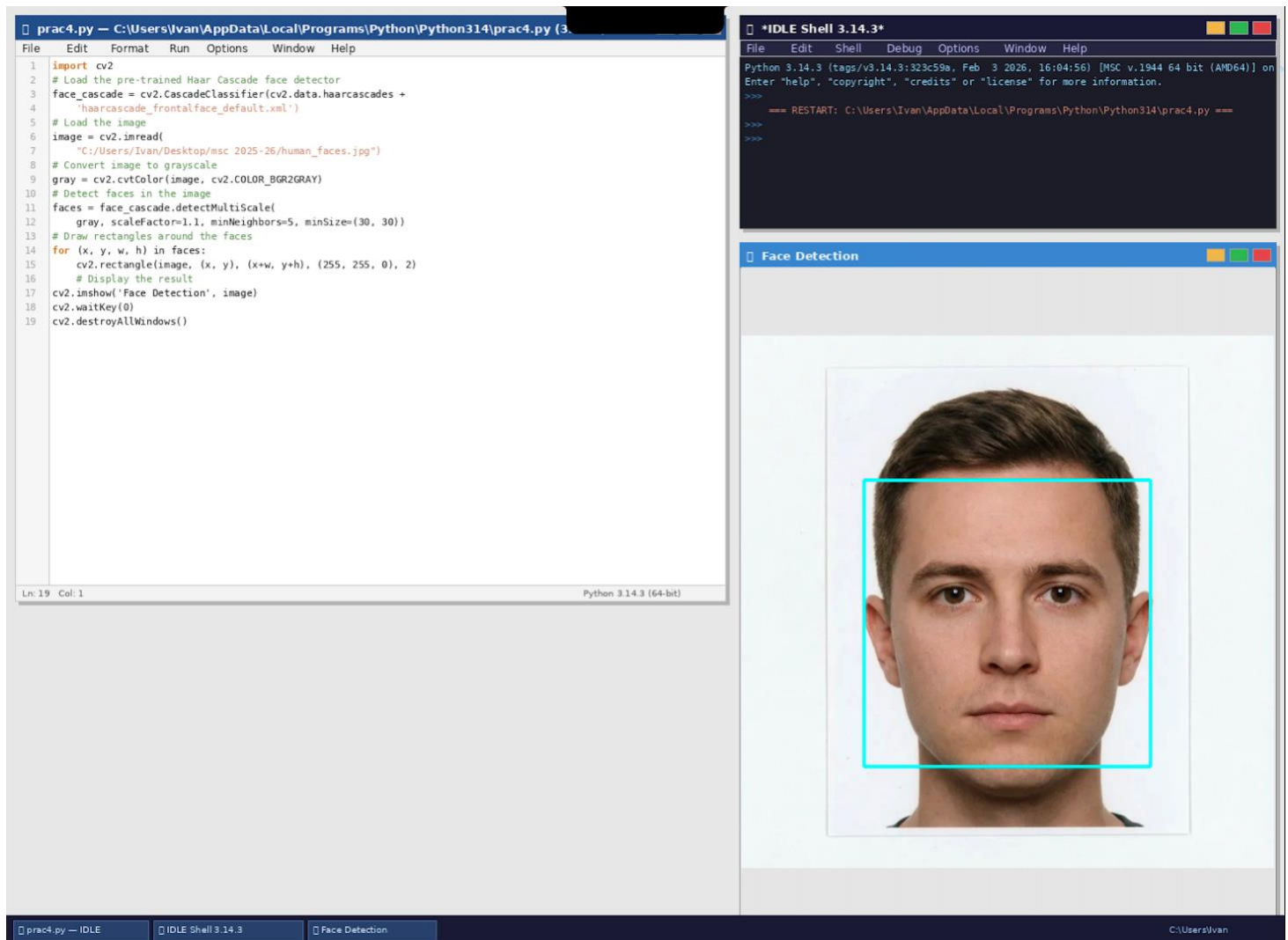
```
1 import numpy as np
2 import cv2
3 import glob
4 import os
5
6 # Define the number of corners in the chessboard
7 num_corners_x = 9
8 num_corners_y = 6
9 # Prepare object points
10 objp = np.zeros((num_corners_x * num_corners_y, 3), np.float32)
11 objp[:, :2] = np.mgrid[0:num_corners_x, 0:num_corners_y].T.reshape(-1, 2)
12 # Arrays to store object and image points
13 objpoints = [] # 3d points
14 imgpoints = [] # 2d points
15 # Load images
16 images = glob.glob(
17     "C:/Users/Ivan/AppData/Local/Programs/Python/Python314/calib*.png")
18 img_shape = None
19 for fname in images:
20     img = cv2.imread(fname)
21     if img is None:
22         print(f"Could not read image {fname}")
23         continue
24     gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
25     if img_shape is None:
26         img_shape = gray.shape[::-1]
27     ret, corners = cv2.findChessboardCorners(gray,
28         (num_corners_x, num_corners_y), None)
29     if ret:
30         objpoints.append(objp)
31         corners2 = cv2.cornerSubPix(gray, corners, (11,11), (-1,-1),
32             criteria=(cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30, 0.001))
33         imgpoints.append(corners2)
34         img = cv2.drawChessboardCorners(img,
35             (num_corners_x, num_corners_y), corners2, ret)
36         cv2.imshow('img', img)
37         cv2.waitKey(500)
38     else:
39         print(f"Chessboard not detected in image: {fname}")
40 cv2.destroyAllWindows()
```

IDLE Shell Output:

```
Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:56) [MSC v.1944 64bit] on
>>>
== RESTART: C:/Users/Ivan/AppData/Local/Programs/Python/Python314/prac3.py ==
>>>
Camera matrix:
[[4.64469618e+02  0.00000000e+00  1.15964406e+02]
 [0.00000000e+00  7.26858621e+02  1.05734067e+02]
 [0.00000000e+00  0.00000000e+00  1.00000000e+00]]
Distortion coefficients:
[[-1.36935775  19.8752062  0.27662763 -0.18081752 -78.87862164]]
>>>
```

Image Window (img): A chessboard image with detected corners and lines connecting them, illustrating the calibration process.

Practical 4 — Face Detection



Practical 5 — Object Detection (TensorFlow SSD Mobile Net)

pracs5.py — C:\Users\Ivan\AppData\Local\Programs\Python\Python314\pracs5.py (3.14.3)

File Edit Format Run Options Window Help

```
1 import numpy as np
2 import os
3 import tensorflow as tf
4 import cv2
5
6 # Load the pre-trained model
7 MODEL_NAME = 'ssd_mobilenet_v2_coco_2018_03_29'
8 PATH_TO_CKPT = os.path.join(MODEL_NAME, 'frozen_inference_graph.pb')
9 NUM_CLASSES = 90
10
11 detection_graph = tf.Graph()
12 with detection_graph.as_default():
13     od_graph_def = tf.compat.v1.GraphDef()
14     with tf.io.gfile.GFile(PATH_TO_CKPT, 'rb') as fid:
15         serialized_graph = fid.read()
16         od_graph_def.ParseFromString(serialized_graph)
17         tf.import_graph_def(od_graph_def, name='')
18
19 # Load label map
20 PATH_TO_LABELS =
21     "C:/Users/Ivan/Desktop/msc_2025-26/python software/mscoco_label_map.pbtxt"
22 category_index = {}
23 with open(PATH_TO_LABELS, 'r') as f:
24     lines = f.readlines()
25     for line in lines:
26         if 'id:' in line:
27             id_index = int(line.strip().split(':')[1])
28         if 'display_name:' in line:
29             name = line.strip().split(':')[1].strip().strip('\"')
30             category_index[id_index] = {'name': name}
31
32 # Function to perform object detection
33 def detect_objects(image):
34     with detection_graph.as_default():
35         with tf.compat.v1.Session(graph=detection_graph) as sess:
36             image_expanded = np.expand_dims(image, axis=0)
37             image_tensor = detection_graph.get_tensor_by_name('image_tensor:0')
38             boxes = detection_graph.get_tensor_by_name('detection_boxes:0')
39             scores = detection_graph.get_tensor_by_name('detection_scores:0')
40             classes = detection_graph.get_tensor_by_name('detection_classes:0')
```

Ln: 58 Col: 1 Python 3.14.3 (64-bit)

IDLE Shell 3.14.3

File Edit Shell Debug Options Window Help

```
Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:56) [MSC v.1944 64bit] on win32
Enter "help", "copyright", "credits" or "license()" for more information.
>>>
=== RESTART: C:\Users\Ivan\AppData\Local\Programs\Python\Python314\pracs5.py ===
>>>
Detected: stop sign (0.94), car (0.87), person (0.81)
>>>
```

Object Detection

C:\Users\Ivan



Practical 7 — Feature Extraction using RANSAC

prac7.py — C:\Users\Ivan\AppData\Local\Programs\Python\Python314\prac7.py (3.14.3)

File Edit Format Run Options Window Help

```
1 import numpy as np
2 from sklearn.linear_model import RANSACRegressor
3 import matplotlib.pyplot as plt
4
5 # Generate some noisy data points
6 np.random.seed(0)
7 x = np.random.uniform(0, 10, 100)
8 y = 2 * x + 1 + np.random.normal(0, 1, 100)
9
10 # Add outliers
11 outliers_index = np.random.choice(100, 20, replace=False)
12 y[outliers_index] += 10 * np.random.normal(0, 1, 20)
13
14 # Stack the points for RANSAC
15 data = np.vstack((x, y)).T
16
17 # Initialize RANSAC model (using sklearn)
18 ransac = RANSACRegressor()
19
20 # Fit the model
21 ransac.fit(data[:, 0].reshape(-1, 1), data[:, 1])
22
23 # Extract the inliers and outliers
24 inlier_mask = ransac.inlier_mask_
25 outlier_mask = np.logical_not(inlier_mask)
26
27 # Extract the line parameters
28 line_slope = ransac.estimator_.coef_[0]
29 line_intercept = ransac.estimator_.intercept_
30
31 # Plot the data points
32 plt.scatter(data[inlier_mask][:, 0], data[inlier_mask][:, 1],
33             c='b', label='Inliers')
34 plt.scatter(data[outlier_mask][:, 0], data[outlier_mask][:, 1],
35             c='r', label='Outliers')
36
37 # Plot the fitted line
```

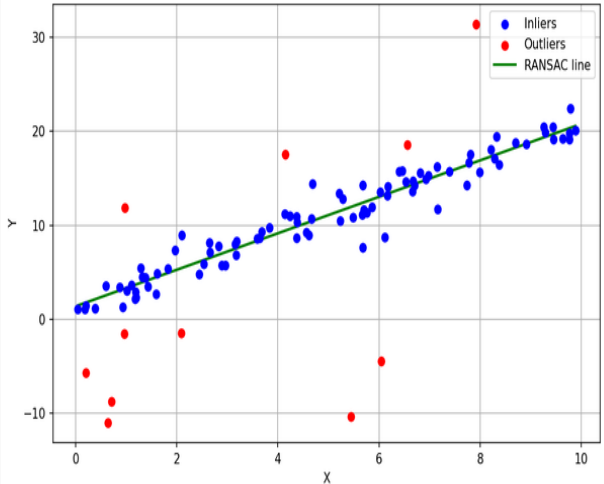
Ln: 43 Col: 1 Python 3.14.3 (64-bit)

IDLE Shell 3.14.3

File Edit Shell Debug Options Window Help

Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:56) [MSC v.1944 64bit] on win32
Enter "help", "copyright", "credits" or "license()" for more information.
>>>
=== RESTART: C:\Users\Ivan\AppData\Local\Programs\Python\Python314\prac7.py ===
>>>
Slope: 1.9403 Intercept: 1.3787
Inliers: 89 Outliers: 11
>>>

Figure 1



C:\Users\Ivan



